

PROVA PRÁTICA I – PARTE I [T1]

Objetivo

Desenvolver uma aplicação *backend* utilizando Spring Boot para disponibilizar uma API REST voltada ao gerenciamento de pessoas, veículos e viagens. A aplicação deve atender aos requisitos de persistência, validação e exposição de dados por meio de DTOs, seguindo uma arquitetura em camadas (Figura 01) e utilizando JPA para o mapeamento objeto-relacional.

Camadas da Aplicação

- Implemente a aplicação seguindo a arquitetura em camadas (Figura 01):
 - Repositório: interfaces estendendo *JpaRepository* para acesso ao banco de dados.
 - Serviço: classes contendo a lógica de negócio da aplicação.
 - Controle: classes responsáveis por expor os *endpoints*.

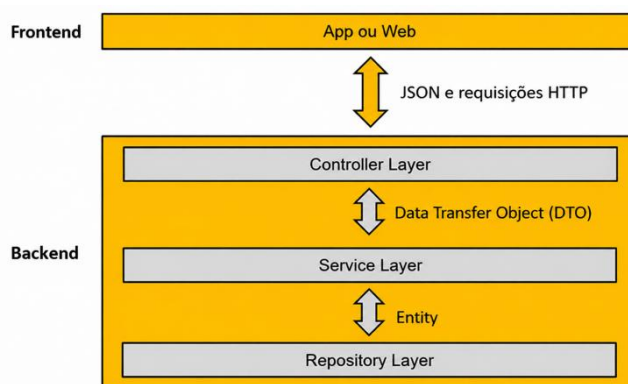


Figura 01: Arquitetura em Camadas

Modelo de Dados

```
public class Pessoa {  
    private Long id;  
    private String nome;  
    private String cpf;  
}
```

```
public class Veiculo {  
    private Long id;  
    private String placa;  
    private String descricao;  
    private Pessoa proprietario;  
}
```

```
public class Viagem {  
    private Long id;  
    private Situacao situacao;  
    private Pessoa passageiro;  
    private Veiculo veiculo;  
}
```

```
public enum Situacao {  
    EM_ANDAMENTO,  
    FINALIZADA  
}
```

Requisitos de Modelagem

- Pessoa
 - Todos os campos são obrigatórios.
 - O campo **cpf** deve ser único.
- Veículo
 - Todos os campos são obrigatórios.
 - O campo **placa** deve ser único.
 - Cada **veículo** deve estar vinculado a um **proprietário** (Pessoa) por meio de um relacionamento **muitos-para-um**.

- Viagem
 - Todos os campos são obrigatórios.
 - Cada **viagem** deve estar associada a um **passageiro** (Pessoa) por meio de um relacionamento **muitos-para-um**.
 - Cada **viagem** deve estar associada a um **veículo** por meio de um relacionamento **muitos-para-um**.

Data Transfer Objects (DTO)

- Crie classes DTO para as entidades com o objetivo de evitar a exposição de dados sensíveis e de objetos complexos. Os DTOs devem seguir as seguintes regras:
 - **Veículo**: além de seus próprios atributos, deve conter apenas o **ID** do **proprietário**.
 - **Viagem**: além de seus próprios atributos, deve conter apenas os **IDs** do **passageiro** e do **veículo** associados.

Regras e restrições

- Realize as implementações e alterações necessárias para atender às seguintes regras de negócio:
 - **R01 - Situação inicial da viagem**: Toda nova viagem cadastrada deve possuir a **situação** definida como em andamento (**Situacao** = **EM_ANDAMENTO**).

Endpoints

- A API deve disponibilizar os seguintes *endpoints* para cada entidade:
 - Pessoa

Método	URL	Descrição
GET	http://localhost:8080/pessoas	Retorna a lista de todas as pessoas
GET	http://localhost:8080/pessoas/{id}	Retorna os dados de uma pessoa pelo ID
POST	http://localhost:8080/pessoas	Cadastra uma nova pessoa
PUT	http://localhost:8080/pessoas/{id}	Atualiza os dados de uma pessoa
DELETE	http://localhost:8080/pessoas/{id}	Remove uma pessoa

- Veículo

Método	URL	Descrição
GET	http://localhost:8080/veiculos	Retorna a lista de todos os veículos
GET	http://localhost:8080/veiculos/{id}	Retorna os dados de um veículo pelo ID
POST	http://localhost:8080/veiculos	Cadastra um novo veículo
PUT	http://localhost:8080/veiculos/{id}	Atualiza os dados de um veículo
DELETE	http://localhost:8080/veiculos/{id}	Remove um veículo
GET	http://localhost:8080/veiculos/proprietario/{proprietarioid}	Retorna a lista de veículos do proprietário informado

- Viagem

Método	URL	Descrição
GET	http://localhost:8080/viagens	Retorna a lista de todas as viagens
GET	http://localhost:8080/viagens/{id}	Retorna os dados de uma viagem pelo ID
POST	http://localhost:8080/viagens	Cadastra uma nova viagem
GET	http://localhost:8080/viagens/veiculo/{veiculoid}	Retorna a lista de viagens do veículo informado
DELETE	http://localhost:8080/viagens/{id}	Remove uma viagem

- Observação:
 - Apenas a situação (**FINALIZADA**) da viagem pode ser alterada pelo usuário. O discente poderá implementar essa alteração de duas formas:

PUT	http://localhost:8080/viagens/{id}	Atualiza apenas a situação da viagem para FINALIZADA .
PATCH	http://localhost:8080/viagens/{id}/situacao	Atualiza a situação da viagem para FINALIZADA .

Carga Inicial do Banco de Dados

- A aplicação deve realizar a carga inicial do banco de dados por meio do arquivo **import.sql**, contendo, no mínimo, **4 registros** em cada tabela da aplicação.

Criação do projeto

- Cada discente deve **criar um novo projeto** utilizando seu **número de matrícula** como parte da identificação nos metadados do projeto (Figura 02).
 - Por exemplo: **prova01_2020113002026**

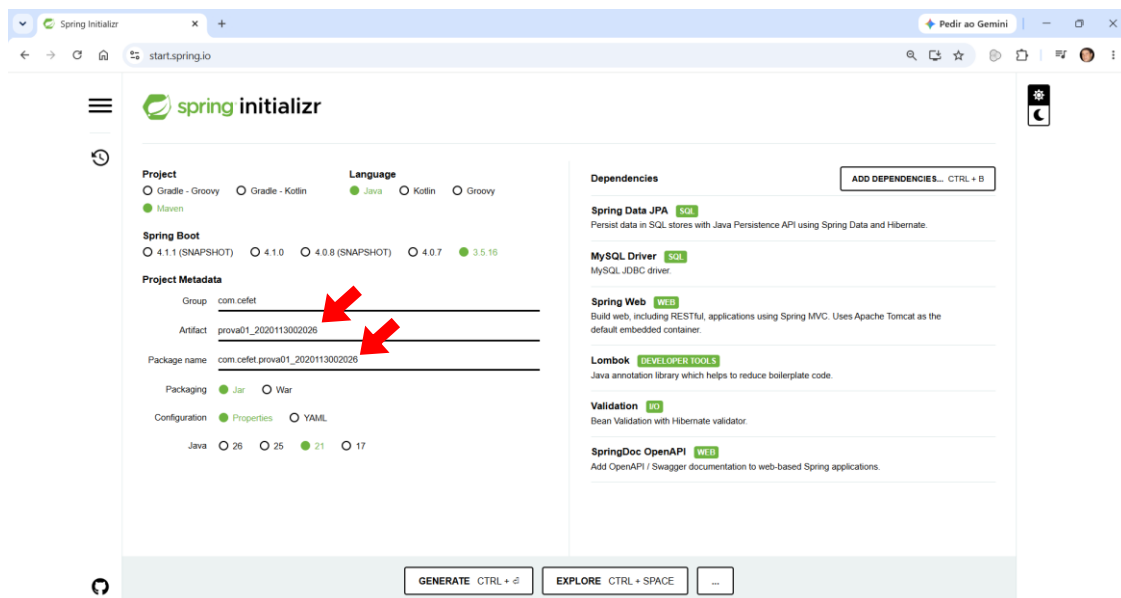


Figura 02: Identificação do projeto por meio do número de matrícula nos metadados

- **ATENÇÃO**
 - Projetos nomeados fora do padrão especificado estarão sujeitos à **penalização de 5 pontos**.

Critérios de avaliação

- Parte I
 - Implementação do arquivo **import.sql** para carga inicial do banco de dados **[1.5 pontos]**
 - Implementação dos **endpoints** da entidade Pessoa **[1.5 pontos]**
 - Implementação dos **endpoints** da entidade Veículo **[1.5 pontos]**
 - Implementação dos **endpoints** da entidade Viagem **[1.5 pontos]**
- Parte II
 - Os requisitos e os critérios de avaliação referentes à Parte II serão divulgados na aula do dia **07/07**.
- **Observação**
 - Os testes e a correção do funcionamento dos **endpoints** serão realizados por meio de um arquivo do Insomnia disponibilizado pelo professor no sistema acadêmico.